

プログラミング的思考 のススメ

魔術学舎United 基盤系5限目



今日の目標



- ❑ プログラミング的思考を知ろう
- ❑ 「当たり前」を定義する方法を学ぼう
- ❑ プログラミング的思考を実践してみよう



01 プログラミング的思考を知ろう

プログラミング的思考とは？



自分が意図する一連の活動を実現するために、**どのような動きの組合せが必要**であり、
一つ一つの動きに対応した記号を、**どのように組み合わせたらいいのか**、
記号の組合せを**どのように改善していけば**、より意図した活動に近づくのか、
といったことを**論理的に考えていく力**



引用元：[小学校段階におけるプログラミング教育の在り方について（議論の取りまとめ）](#)

例えば...



動画で見てみよう！

動画タイム

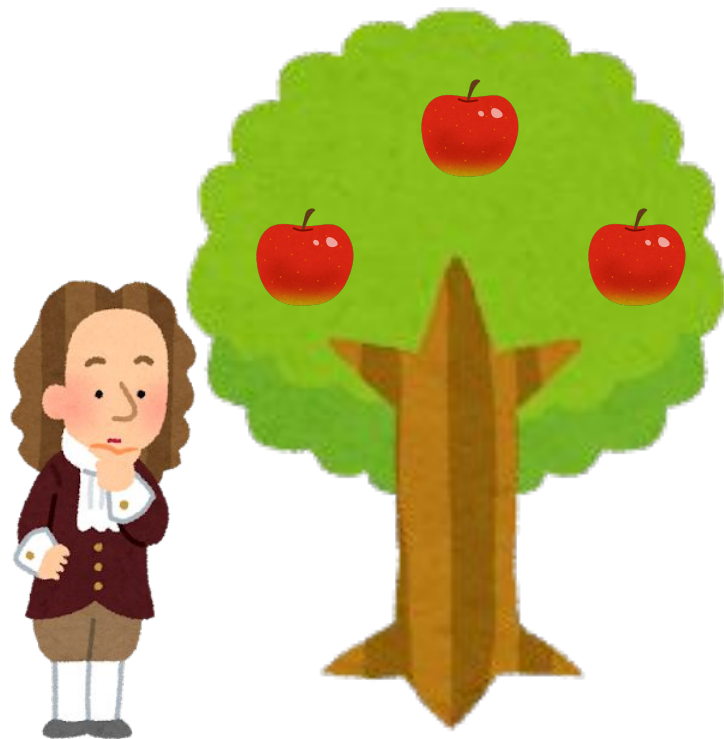
ラムダ技術部

【事故】理系がレシピ通りに料理したら大変な結果に.....

プログラミング的思考とは？



たとえば...

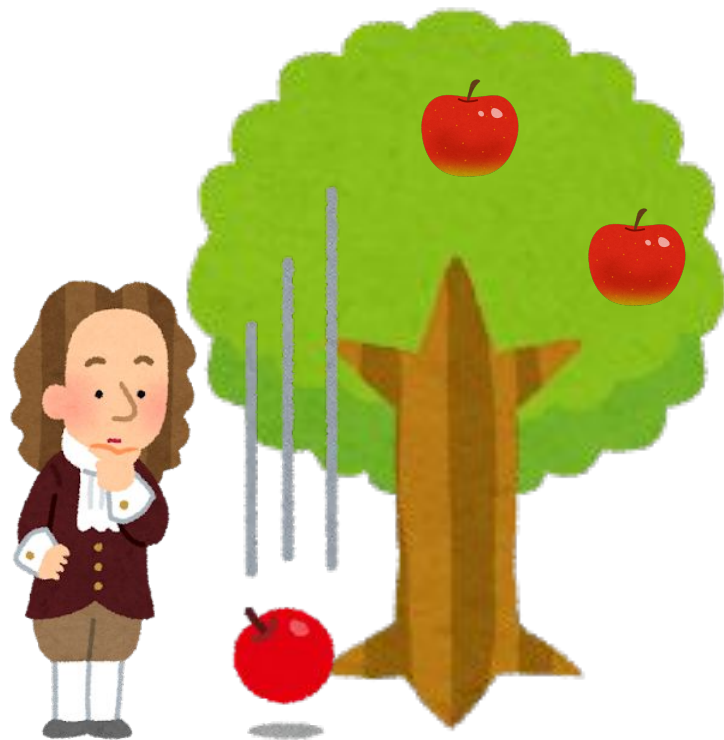


プログラミング的思考とは？



たとえば...

現実世界では
リンゴは勝手に下に落ちます



当然のことですね

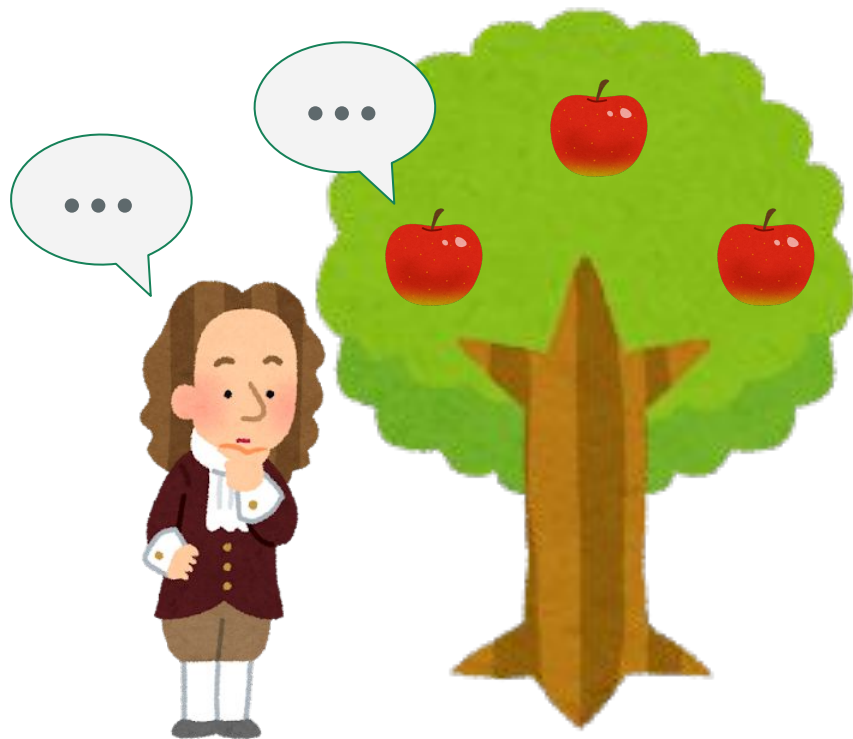
プログラミング的思考とは？



たとえば...

現実世界では
リンゴは勝手に下に落ちますが

プログラミングの世界では
『下に落ちろ』と言わない限り
りんごが勝手に落ちることはない...



プログラミングの世界 = Unityの世界

プログラミング的思考とは？

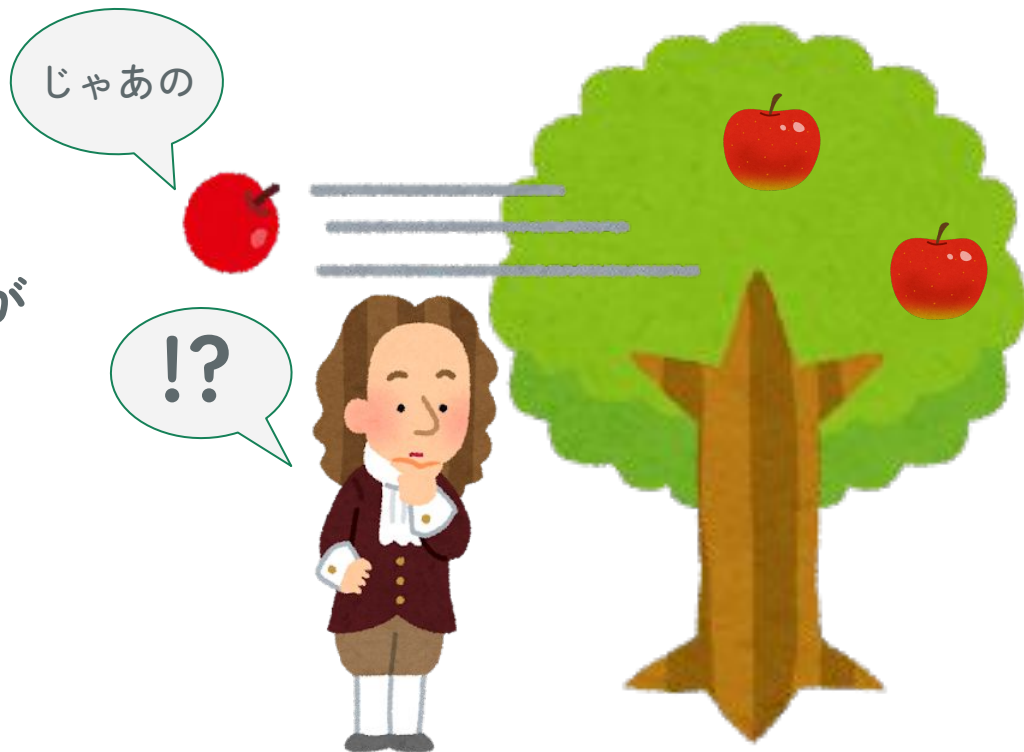


たとえば...

現実世界では
リンゴは勝手に下に落ちますが

プログラミングの世界では
『リンゴが横に落ちる』
という事さえできてしまう

※『横に落ちろ』と命令すれば。



プログラミングの世界 = Unityの世界

プログラミング的思考とは？



重要なことは

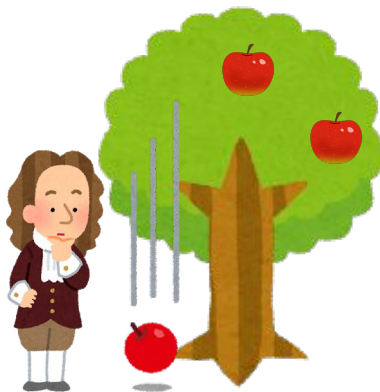


プログラミング的思考とは？



重要なことは

「当たり前」ということが
「当たり前」に動くように定義をすること



「当たり前」を定義をしないとそもそも動かなかったり、おかしいことになってしまう

プログラミング的思考とは？



プログラミング的思考とは



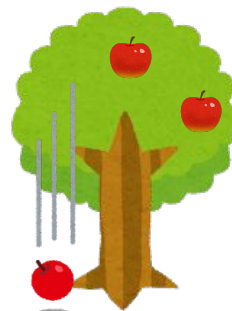
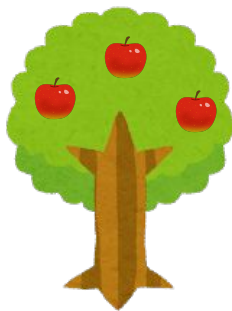
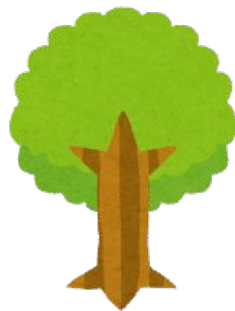
プログラミング的思考とは？



プログラミング的思考とは

自分の作りたい物を実現する為に、

必要なステップを論理的に考えて組み立てる力



お手元の質問ボタンを押してみてください



みなさん、どう押しましたか？
押した結果、どうになりましたか？



ボタンを押す動作ひとつ取っても
必要なステップは沢山ありますね

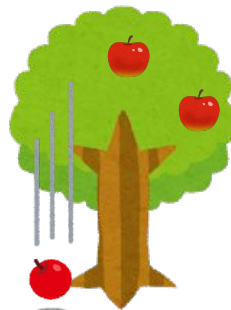
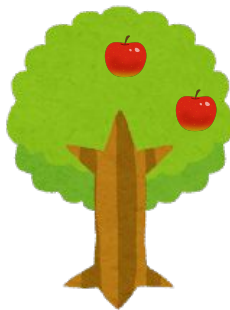


1. ボタンを見る
2. ボタンまで手を伸ばす
3. ボタンに手が当たる
4. ボタンが青くなる
5. "Use" と表示される
6. コントローラーのトリガーを引く
7. 手元に挙手マークが出る
8. 頭上に挙手マークが出る
9. ボタンから手を放す
10. 諸々表示されなくなる

➡ これを論理的に組み立てるのも プログラミング的思考

この思考を身に着けるためには何が必要？

具体的にどう考えたらいいの？



「当たり前」を定義する方法を学ぼう



3つの手順で「当たり前」を定義しよう



分解



一般化



組合せ

「当たり前」を定義する方法を学ぼう



3つの手順で「当たり前」を定義しよう



分解 - 実現したいものを**実現可能な小さな要素**に分けよう



一般化 - 実現したい要素とUnityの機能との**類似性**を見つけよう



組合せ - 機能を**目的に合わせて**組み合わせよう

分解 - 実現可能な小さな要素に分けよう



分解とは - 実現可能な小さな要素に分けよう



プリンを食べるギミックを作りたい！

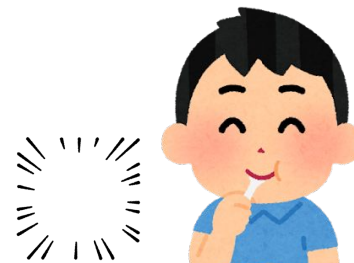
分解とは - 実現可能な小さな要素に分けよう

プリンを食べるギミックを作りたい！

↳ スプーンで一口食べる？



↳ お皿ごとパクッと食べる？



プリンをスプーンでひと口食べる を選ぶとする

プリンをスプーンでひと口食べる を選ぶとする

↳ ほかの人も食べれる？

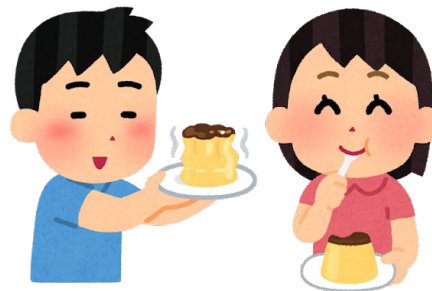
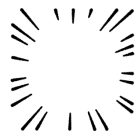
↳ プリンは減らしたい？



分解とは - 実現可能な小さな要素に分けよう

具体的にするほど、作りやすいし、人に聞きやすい！

→できるだけ「細かく」分解してみよう！



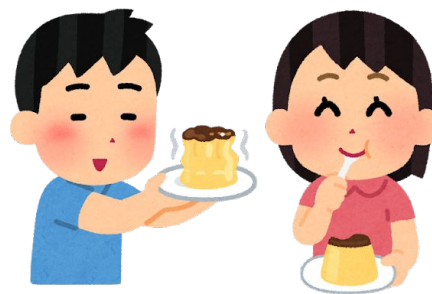
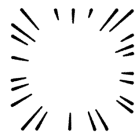
できるできないを考えるのは後でもいい！

★ 分解とは？

分解とは

実現したいものを具体的に考えて

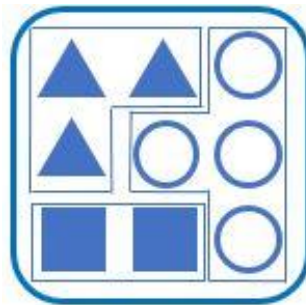
実現しやすい細かい要素に分けること



「当たり前」を定義する方法を学ぼう



一般化 - 要素と機能との類似性を見つけよう



一般化とは - ●●●●への一般化の例



- ・ 帽子の着脱
- ・ ヘイローの回転・停止
- ・ 音楽の再生・停止
- ・ ビームの発射

一般化とは - ●●●●への一般化の例



- ・ 帽子の着脱 ⇒ 被る or 脱ぐ
- ・ ヘイローの回転・停止 ⇒ 回る or 止める
- ・ 音楽の再生・停止 ⇒ 流す or 止める
- ・ ビームの発射 ⇒ 撃つ or 撃たない

一般化とは - ●●●●への一般化の例



- ・ 帽子の着脱 ⇒ 被る or 脱ぐ
- ・ ヘイローの回転・停止 ⇒ 回る or 止める
- ・ 音楽の再生・停止 ⇒ 流す or 止める
- ・ ビームの発射 ⇒ 撃つ or 撃たない

どれも オン or オフ の状態がある

一般化とは - Boolへの一般化の例



- ・ 帽子の着脱 ⇒ 被る or 脱ぐ
- ・ ヘイローの回 ⇒ 回る or 止める
- ・ 音楽の再生・停止 ⇒ 止める
- ・ ビームの発射 ⇒ 撃つ or 撃たない

どれも オン or オフ の状態がある

一般化とは - $\times\times\times$ への一般化の例



次の場合は...？

- ・ じゃんけん
- ・ 信号
- ・ エレベーター
- ・ 干支

一般化とは - ×××への一般化の例



次の場合は...？

- ・じゃんけん ⇒ グー or チョキ or パー
- ・信号 ⇒ 青 or 黄 or 赤
- ・エレベーター ⇒ 1階, 2階, 3階 . . .
- ・干支 ⇒ 子, 丑, 寅, 卯 . . .

一般化とは - ×××への一般化の例



次の場合は...？

- ・じゃんけん ⇒ グー or チョキ or パー
- ・信号 ⇒ 青 or 黄 or 赤
- ・エレベーター ⇒ 1階, 2階, 3階・・・
- ・干支 ⇒ 子, 丑, 寅, 卯・・・

どれも 3種類以上の状態がある

一般化とは - Intへの一般化の例



次の場合は...?

- じゃんけん ⇒ グー or チョキ or パー
- 信号 ⇒ 青 or 黄 or 赤
- エレベーター ⇒ 上 or 下 or 止まる or ...
- 干支 ⇒ 子, 丑, 寅, 卯, ...

Intへの一般化

どれも 3種類以上の状態 がある



一般化とは？



Boolへの一般化

Intへの一般化

一般化とは

実現したい要素とUnityの機能との間に

共通する性質や法則などを見出すこと

「当たり前」を定義する方法を学ぼう



組合せ - 機能を目的に合わせて組み合わせよう

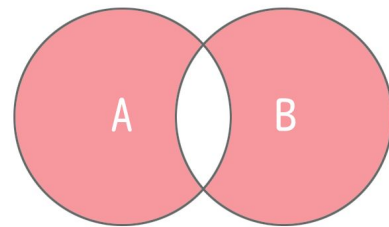
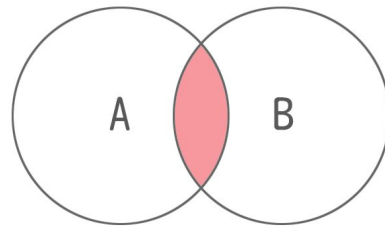


組合せとは？ - 2種類の考えが必要

実現したい複数の要素に対して
目的を考え直したとき

- ・ 前提条件は必要か、不要か
- ・ 状態は並列か、排他か

状態1 $\Rightarrow$ 状態2



組合せとは？ - 前提条件を考えよう



- ・ 前提条件は必要か、不要か

銃を撃つために、銃を持っている必要がある

銃を撃つために、帽子を被っている必要はない



組合せとは？ - 前提条件を考えよう



前提条件が**必要**ということは、

順番の前後が入れ替わってはいけないという事

組合せとは？ - 前提条件を考えよう



前提条件が**必要**ということは、

順番の前後が入れ替わってはいけないという事

銃を持つ前に撃つことはできない



「銃を持つ」と「銃を撃つ」には明らかな順番がある



組合せとは？ - 前提条件を考えよう



前提条件が**不要**ということは、

どのタイミングで実行しても大丈夫という事

組合せとは？ - 前提条件を考えよう



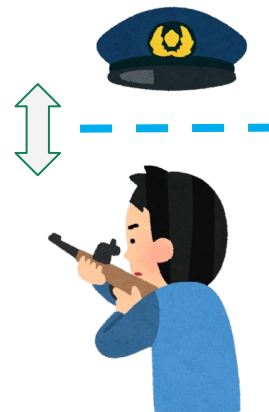
前提条件が**不要**ということは、

どのタイミングで実行しても大丈夫という事

銃を撃つために帽子を被っている必要はない



「銃を撃つ」と「帽子を被る」という動作は独立している

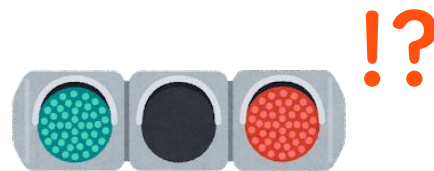


組合せとは？ - 並列排他を考えよう



・ 状態は並列か、排他か

片手でピースする時もあれば両手でする時もある
信号のランプが2個同時に点くことはない



組合せとは？ - 並列排他を考えよう



状態が**並列**であるということは、

複数の状態が同時に起こりうるという事

組合せとは？ - 並列排他を考えよう



状態が**並列**であるということは、

複数の状態が同時に起こりうるという事

「左手でピース」した上で「右手もピース」ができる



互いの状況が衝突していない



組合せとは？ - 並列排他を考えよう



状態が**排他**であるということは、

複数の状態が同時に起こりえないという事

組合せとは？ - 並列排他を考えよう



状態が**排他**であるということは、

複数の状態が同時に起こりえないという事

「信号が青である」と「信号が赤である」は両立できない



互いの状態が衝突している



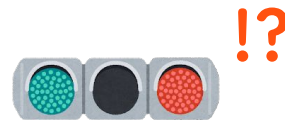
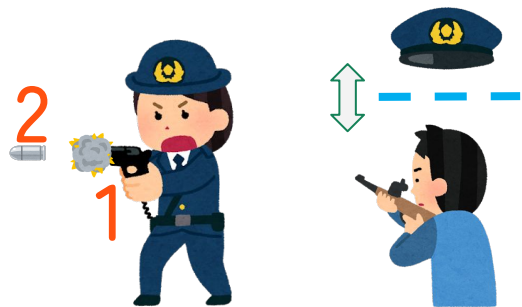
!?

★ 組合せとは？

組合せとは 目的に合わせて以下を考えて機能を組むこと

前提条件は必要か、不要か

状態は並列か排他か





02

プログラミング的思考を実践してみよう

プログラミング的思考を実践してみよう



【グループワーク 5分】

- ・ 2人1組になって「当たり前」を定義してみよう



□ 懐中電灯のギミックで何を実現したい？



分解 - 実現したいものを **実現可能な小さな要素**に分けよう



一般化 - 実現したい要素とUnityの機能との **類似性**を見つけよう



組合せ - 機能を **目的に合わせて**組み合わせよう

話してみてどうでしたか？

- ・ 正解は一つじゃない
- ・ 他の人と話してみると意外な方法が出てくるかも

★ 今日のまとめ



- ❑ 「当たり前」のことをしっかりと定義しよう
- ❑ 分解、一般化、組合せの考え方を駆使しよう
- ❑ 人と話すのも大切。ギミックの実現手順は1つではない！



One More Step!

プログラミング的思考 -上級編-

因果関係と相関関係について

- ・ 原因⇒結果の一方通行が因果関係
- ・ 2つの内容が互いに影響するのが相関関係

One More Step!

因果関係

雨が降ると傘が多く売れる

⇒傘を多く売っても雨が降るわけではない





One More Step!

相関関係

雨が降っているときには傘が多く売れている

⇒傘が売れてないときは雨は降ってない



さっきと同じ文章に見える？よ〜く読んでください

プログラミング的思考のススメ

魔術学舎United 基盤系5限目

